Complementary design notes | AI Opportunity Analyser | BDO CASE

BDO CASE | AI Engineer | Dr. Ravindra R. Patil

This document expands on the decisions stated briefly in the README and documents the journey behind the delivered solution.

Design decisions: choice, reasoning, rejected alternative

Decision	Chosen	Why	Rejected alternative
Missing website in Brreg	Record registry truth as a fact; show an explicit 'unavailable' status; continue with registry facts	An incomplete but clearly labelled analysis beats a crash; generality over test-fitting	Hard-coded site hints (breaks generality); a web-search fallback was later added as an opt-in, verified option (see below)
Source selection rule	A search URL is eligible only when its domain carries a token of the company's own name (generic words stripped); companies without a distinctive token can qualify via the orgnr check	Allowlist by construction: social media, aggregators and directories are excluded automatically, with no blocklist to maintain	A blocklist of third-party sites: every new aggregator (LinkedIn, Yahoo Finance appeared in real results) would need a new entry
Blocked site handling	Retry the direct fetch with two browser identities, then fetch the same URL through the Reader service; the text must still pass verification	Some blocks are identity-based (a second identity helps), some are IP-based (only different infrastructure helps)	Residential proxies or scraping services: defeats an explicit block, wrong for an audit firm's tool
Citation scheme	Registered homepage content is cited as brreg:website; a search-verified page as websearch:text; a web-only suggestion without a supporting registry fact is rejected	Every citation shows its true origin; no web-derived claim can hide behind a Brreg fact	Auto-adding a registry citation to pass validation: would make the provenance dishonest
Platform	Render.com, Docker, free tier, Frankfurt	Git-connected deploy, no credit card, EU region; same image runs locally	Azure Container Apps: prepared but blocked by the ACR Tasks restriction for new subscriptions
LLM provider / model	Groq free tier, gpt-oss-20b (OpenAI-compatible)	Right size for a small structured task; JSON mode; free; provider behind configuration	gpt-oss-120b (heavier, no gain at this scope); compound (web search violates grounding); llama 3.x (retired Aug 2026)
Access protection	Public demo, unprotected by design	Public registry data; per-client rate limit and bounded cost; optional shared token exists	Token-by-default or full auth: unnecessary for public data at demo scope; Entra ID listed for production
Storage	SQLite keyed by orgnr	Zero installation, right size; swap interface exists for PostgreSQL later	Managed PostgreSQL now: cost and complexity without demo benefit
Fixed format	Strict JSON, extra='forbid', validated before use	Only format code can validate honestly: schema, fact-ID citations and assessment flag are machine-checked	Free prose: trust would become guesswork
Expected values	Qualitative only; the prompt forbids invented percentages, figures, dates or targets in titles, descriptions and expected values	Unsourced specifics (e.g. 'net-zero by 2050') destroy trust in an audit context	Quantified benefits: impressive in a demo, indefensible in a review
Abuse protection	MOD11 check, cache, per-client rate limit (10/h), strict schema	Layered, cheapest checks first; repeats cost nothing	Gateway/redis limits: multi-instance production need, not demo
Data rules	Registry + verified own-site content only; no board members or personal data; logs contain orgnr, sources, model	Public information only; privacy by omission at the collection point	Storing personal fields 'in case': violates the data rules and adds risk

Consultant view	The page shows only sources the analysis used (ok status); rejected attempts stay in the saved data and logs	The page's motive is trust in the result, not a log of attempts; auditability is preserved in the logs	Displaying every rejected candidate: noise for the consultant, no added trust

What did not work at first

• GitHub Models (gpt-4o-mini), the planned free provider, was fully retired by Microsoft on 30 July 2026, mid-project; the endpoint returned stub responses to all callers. The switch to Groq required no code change.

• Azure Container Apps deployment was fully prepared but blocked by Azure's restriction on ACR Tasks for new subscriptions; the deployment moved to Render, keeping the free tier.

• Early model runs occasionally returned valid JSON with zero use cases or markdown-fenced answers; each was rejected by validation, retried with a corrective instruction, and the prompt was hardened.

• Real search results for the test companies included LinkedIn and Yahoo Finance pages; verifying them against the registry facts was not enough, because a fact-check alone could let an aggregator pass. This led to the general company-domain rule.

• Solstad's site blocks datacenter IPs with 403 even for browser identities; the Reader service rung was added so the live page could still be fetched from different infrastructure, and the fetched text needed consent-banner cleaning before it was useful.

What I would add with more time

• A prompt-evaluation set covering all five test companies, scored against a checklist, tracked across prompt versions.

• Durable cloud storage for the cache, and multi-instance rate limiting.

• Operational telemetry: request metrics, LLM cost tracking, alerting.

The agentic next step: how the next version scales into agents

Important Note *- The case title names the product vision ('AI-agent som forbereder salgsmuligheter'); the current version delivers that vision as a grounded, deterministic pipeline. The architecture follows the data: at this scope (one registry record plus one unstructured source: the company website), a fixed pipeline is the right tool; as the data grows into a real per-company corpus, vector retrieval and agent orchestration become the right fit. The design below is the concrete next step, built on the same trust contract whose foundation is already delivered here: fact-ID grounding, visible source statuses, separate facts/assessment zones, and provenance logging.*

• **Tool layer:** the LLM gains callable tools - a Brreg lookup tool, a web-fetch tool, and a document-retrieval tool over a per-company corpus (vector store) - so it gathers evidence itself instead of receiving one fixed fetch.

• **Planning loop:** an orchestration loop (LangGraph or a hand-rolled state machine) in which the LLM decides the next step, observes the result, and iterates until it has enough evidence for an answer.

• **Verification step:** a second LLM pass checks each suggested use case against the retrieved documents (the quality-check agent loop), keeping every suggestion verifiable even when built from sources beyond the registry.

• **Guardrails:** per-run step and cost budgets, trace logging of every agent decision (which tool, which source, which model), and an evaluation harness, because an agent path is no longer deterministic.

Technical requirements compliance

Requirement	Fulfilled by
Cloud native, URL, platform justified, free tier	Docker container deployed on Render free tier (EU); platform justification in the deployment doc and the choices table
Describe the cloud setup; IaC is a plus	infra/README_deployment.md describes every step; render.yaml is the infrastructure-as-code blueprint
API keys and secrets not in the code	Keys enter only through environment variables (Render dashboard / .env); .gitignore blocks .env; no key value appears anywhere in the code
Predictable errors from external services	Brreg timeouts, missing websites, blocked pages and oversized responses become visible statuses; LLM answers are validated with one corrective retry; every failure path is tested

Automated tests without external calls	5 offline pytest tests with mocked externals; the conftest guarantee forces offline mode on any machine
Logs show sources and model per analysis	Every analysis logs orgnr, source statuses (brreg, website, websearch) and the model name
Protection against LLM cost misuse	MOD11 input check before any call, per-client rate limit (10 per hour), SQLite cache so repeats cost nothing, schema limits of 2-3 use cases

All requirements above are verified against the delivered code, and the full test suite passes offline.